# RED-BLACK TREE

```
#include <iostream>

using namespace std;

enum Color { RED, BLACK };

struct Node {

	int data;

	Color color;

	Node *left, *right, *parent;

Node(int data) {

		this->data = data;

		left = right = parent = NULL;

		color = RED;

	}

};

class RBTree {

private:

	Node* root;

void rotateLeft(Node* &root, Node* &pt) {

		Node* pt_right = pt->right;

	    pt->right = pt_right->left;

		if (pt->right != NULL)

			pt->right->parent = pt;

		pt_right->parent = pt->parent;

		if (pt->parent == NULL)

			root = pt_right;
```

```
        else if (pt == pt->parent->left)
                pt->parent->left = pt_right;
        else
                pt->parent->right = pt_right;
        pt_right->left = pt;
        pt->parent = pt_right;
}
void rotateRight(Node* &root, Node* &pt) {
        Node* pt_left = pt->left;
        pt->left = pt_left->right;
        if (pt->left != NULL)
                pt->left->parent = pt;
        pt_left->parent = pt->parent;
        if (pt->parent == NULL)
                root = pt_left;
        else if (pt == pt->parent->left)
                pt->parent->left = pt_left;
        else
                pt->parent->right = pt_left;
        pt_left->right = pt;
        pt->parent = pt_left;
}
void fixViolation(Node* &root, Node* &pt) {
        Node *parent_pt = NULL;
        Node *grand_parent_pt = NULL;
```

```
while ((pt != root) && (pt->color != BLACK) &&
        (pt->parent->color == RED)) {
    parent_pt = pt->parent;
    grand_parent_pt = pt->parent->parent;
    // Parent is left child
    if (parent_pt == grand_parent_pt->left) {
        Node *uncle_pt = grand_parent_pt->right;
        // Case 1: Uncle is RED
        if (uncle_pt != NULL && uncle_pt->color == RED) {
            grand_parent_pt->color = RED;
            parent_pt->color = BLACK;
            uncle_pt->color = BLACK;
            pt = grand_parent_pt;
        } else {
            // Case 2: pt is right child
            if (pt == parent_pt->right) {
                rotateLeft(root, parent_pt);
                pt = parent_pt;
                parent_pt = pt->parent;
            }
            // Case 3
            rotateRight(root, grand_parent_pt);
            swap(parent_pt->color, grand_parent_pt->color);
            pt = parent_pt;
        }
```

```
            }
            // Parent is right child
            else {
                Node *uncle_pt = grand_parent_pt->left;
                if ((uncle_pt != NULL) && (uncle_pt->color == RED)) {
                    grand_parent_pt->color = RED;
                    parent_pt->color = BLACK;
                    uncle_pt->color = BLACK;
                    pt = grand_parent_pt;
                } else {
                    if (pt == parent_pt->left) {
                        rotateRight(root, parent_pt);
                        pt = parent_pt;
                        parent_pt = pt->parent;
                    }
                    rotateLeft(root, grand_parent_pt);
                    swap(parent_pt->color, grand_parent_pt->color);
                    pt = parent_pt;
                }
            }
        }
        root->color = BLACK;
    }
    void inorderHelper(Node* root) {
        if (root == NULL)
```

```
            return;
        inorderHelper(root->left);
        cout << root->data << "(" << (root->color == RED ? "R" : "B") << ") ";
        inorderHelper(root->right);
    }
public:
    RBTree() { root = NULL; }
    void insert(const int &data) {
        Node* pt = new Node(data);
        Node* y = NULL;
        Node* x = root;
        while (x != NULL) {
            y = x;
            if (pt->data < x->data)
                x = x->left;
            else
                x = x->right;
        }
        pt->parent = y;
        if (y == NULL)
            root = pt;
        else if (pt->data < y->data)
            y->left = pt;
        else
            y->right = pt;
```

```
        fixViolation(root, pt);
    }
    void inorder() {
        inorderHelper(root);
    }
};
int main() {
    RBTree tree;
    int arr[] = {10, 20, 30, 15, 25, 5};
    for (int x : arr)
        tree.insert(x);
    cout << "Inorder traversal (value(color)):\n";
    tree.inorder();
    return 0;
}
```

Output

```
Inorder traversal (value(color)):
5(R) 10(B) 15(R) 20(B) 25(R) 30(B)

=== Code Execution Successful ===
```